_blexin.

CON NOI È SEMPLICE

Claude Code Workshop



Gerardo Greco
Senior Developer &
Devops Engineer @ Blexin
gerardo.greco@blexin.com



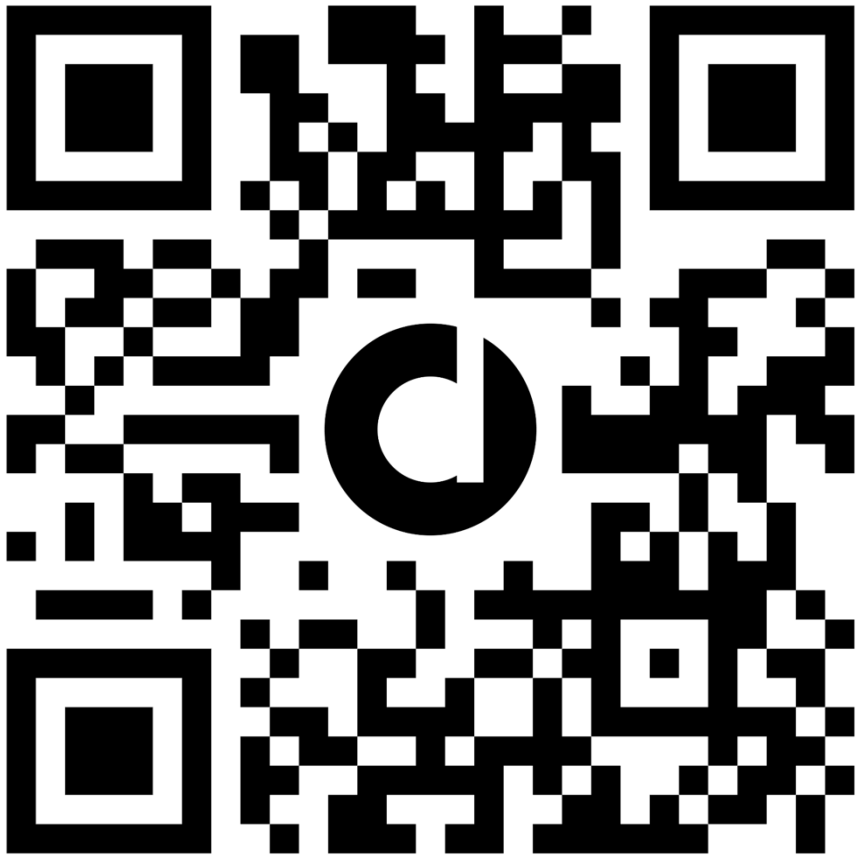
Antonio Liccardi
Chief Technology Officer @ Blexin
Microsoft MVP for Dev Technologies
antonio.liccardi@blexin.com

Come è organizzato il codice?

- Il workshop è strutturato in moduli
- I moduli che includono codice sono organizzati nelle cartelle Esercizio/Soluzione
- Per gli esercizi è possibile scegliere tra codice in C#, TypeScript o Python
- Ogni modulo include il file Readme.md con le istruzioni

CON NOI È SEMPLICE

Repository del workshop



https://git.new/sVnBlLV

CON NOI È SEMPLICE

Il contesto e la sua gestione

Il contesto è la nostra unica vera leva. Darlo giusto, al momento giusto.

Più questa finestra aumenta, maggiore è la probabilità che compaiano due fenomeni:

- **Context rot:** quando il rumore degrada la qualità complessiva delle risposte
- **Needle in a haystack:** quando l'informazione decisiva si perde e non viene recuperata correttamente

CON NOI È SEMPLICE

Token



Un token è una piccola unità di testo: può essere una parola, una parte di parola, un simbolo o un segno di punteggiatura.

CON NOI È SEMPLICE

Tipologie di token

1



Input token

Sono tutti i token che il modello riceve prima di rispondere.

Esempi: system prompt, cronologia, documenti, domanda utente

2



Output token

Sono i token generati dal modello nella risposta.

La risposta viene prodotta un token alla volta.

3



Cached input token

Sono token di input già riutilizzati perché una parte del prompt è identica a una richiesta precedente.

Aiutano a ridurre costo e latenza.



I modelli

Claude Fable 5.1
Massima capacità

- Ricerca e analisi approfondite
- Contesti molto complessi e documenti estesi
- Sessioni agentiche lunghe e pianificazione

Claude Opus 5.5
Lavoro agentico e coding complesso

- Sviluppo e refactoring di sistemi complessi
- Task agentici multi-step
- Problemi che richiedono ragionamento avanzato

Claude Sonnet 5.5
Equilibrio tra velocità e capacità

- Coding quotidiano e sviluppo applicazioni
- Analisi dati e creazione di contenuti
- Buon equilibrio tra qualità, velocità e costo

Claude Haiku 5.5
Massima velocità e costo minimo

- Classificazione, estrazione e routing
- Applicazioni real-time e bassa latenza
- Sub-agent e task ad alto volume

Maggiore capacità e qualità ← Fable 5.1 · Opus 5.5 · Sonnet 5.5 · Haiku 5.5 → Maggiore velocità e minor costo

CLAUDE.md/AGENTS.md

AGENTS.md

```
## Setup commands

* `dotnet restore`
* `dotnet run`
* `dotnet test`

## Code style

* C# with nullable reference types enabled
* Follow standard .NET naming conventions
* Prefer dependency injection and async/await
* Keep code simple and readable
```

Strumento per fornire contesto essenziale al coding agent.

È un markdown a livello root, iniettato nel system prompt a ogni sessione.

Ogni riga costa token: è consigliabile sotto le 500 righe.

```
/init

/doctor prompt-audit
```

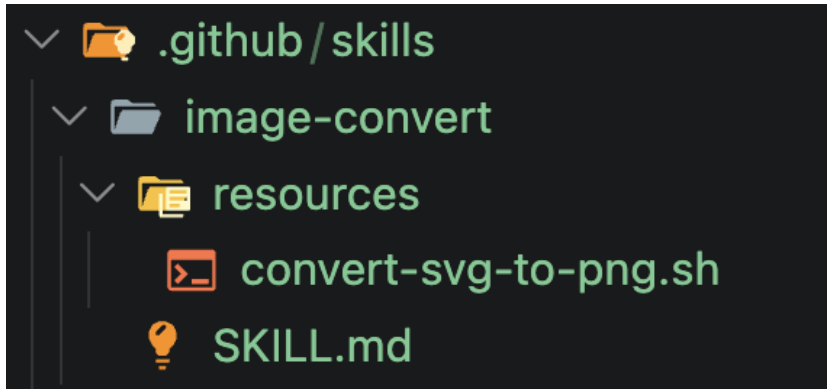
CON NOI È SEMPLICE

Skills

.github/skills/image-convert/SKILL.MD

```
---
name: image-convert
description: Converts SVG images to PNG format.
Use when asked to convert SVG files.
allowed-tools: shell
---

When asked to convert an SVG to PNG, run the
`convert-svg-to-png.sh` script from this skill's
base directory, passing the input SVG file path as the
first argument.
```



Una Skill è una cartella autocontenuta con script e risorse opzionali che fornisce a Copilot una capacità specializzata e portabile.

Si carica automaticamente quando il prompt corrisponde alla sua descrizione.

CON NOI È SEMPLICE

Hands-on

Modulo 1

Server MCP

.copilot/mcp-config.json

```
{
  "mcpServers": {
    "playwright": {
      "type": "local",
      "command": "npx",
      "args": ["@playwright/mcp@latest"],
      "env": {},
      "tools": ["*"]
    },
    "context7": {
      "type": "http",
      "url": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "YOUR-API-KEY"
      },
      "tools": ["*"]
    }
  }
}
```

Un server MCP (Model Context Protocol) è un processo che espone strumenti, risorse e prompt agli agenti AI tramite un protocollo standardizzato. È il modo in cui Copilot accede a database, API, file system e servizi di terze parti senza integrazioni ad hoc.

VS Code funge da client MCP: il server gira localmente o in remoto, configurabile per workspace o globalmente. Una volta connesso, i suoi strumenti sono disponibili agli agenti insieme a quelli nativi.

CON NOI È SEMPLICE

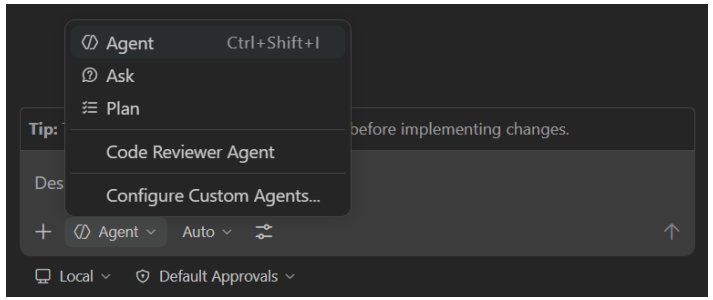
Agenti

.github/agents/reviewer-agent.json

```
---
name: Code Review Agent
tools: [execute/runTests, read, agent, search, todo]
model: GPT-5.4 (copilot)
handoffs:
  - label: Start fixes
    agent: csharp-developer
    prompt: Implement the fixes
    model: GPT-5.4 (copilot)
---
Review the changes in this PR from different angles.
Perform these reviews in parallel:

- Run the security-reviewer agent to check for vulnerabilities
- Run the performance-reviewer agent to identify bottlenecks
- Run the accessibility-reviewer agent to verify a11y compliance

Consolidate findings into a single review summary.
```



I subagenti sono assistenti AI specializzati che gestiscono attività secondarie nel proprio contesto, restituendo soltanto un riepilogo e mantenendo pulita la conversazione principale.

Ciascuno opera con la propria finestra di contesto, prompt di sistema, strumenti e permessi.

CON NOI È SEMPLICE

Hands-on

Modulo 2

Hooks

.github/hooks/hooks.json

```
"userPromptSubmitted": [
  {
    "type": "command",
    "bash": "./scripts/log-prompt.sh",
    "powershell": "./scripts/log-prompt.ps1",
    "cwd": "scripts",
    "env": {
      "LOG_LEVEL": "INFO"
    }
  }
]
```

Gli hook sono comandi shell, endpoint HTTP o prompt LLM definiti dall’utente che vengono eseguiti automaticamente in punti specifici del ciclo di vita di GitHub Copilot.

CON NOI È SEMPLICE

Hands-on

Modulo 3



Il lavoro dura più sessioni, il contesto no



La context window è finita, ma il lavoro reale dura più sessioni. Servono due cose:

- **Recuperare lo stato:** tornare a un punto sicuro quando un esperimento va male
- **Persistere la conoscenza:** convenzioni e decisioni non vanno ricostruite da zero

4 strumenti:

Checkpoints · Memorie
Tokensave · Episodic memory

CON NOI È SEMPLICE

Checkpoint vs Git vs Memoria



Checkpoint

undo di sessione



Git

storia permanente



Memoria

conoscenza in contesto

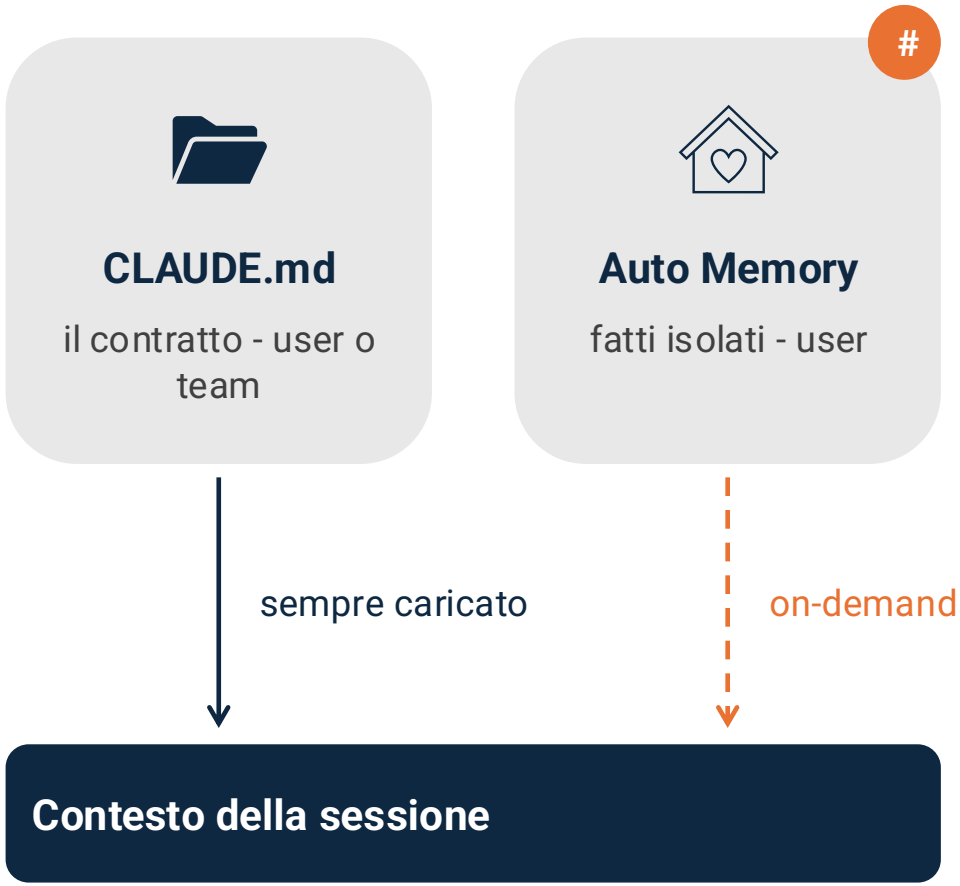
Tre cose diverse, da non confondere:

- **Checkpoint:** undo di sessione: ripristina codice e conversazione
- **Git:** storia permanente: commit, branch, collaborazione
- **Memorie:** non ripristina nulla: conoscenza che resta in contesto

Il checkpoint non sostituisce git.

CON NOI È SEMPLICE

Memorie: scope e meccanismi



Due scope

- **User-level:** ~/.claude/, solo te, su tutti i progetti
- **Project-level:** nel repo, condiviso col team

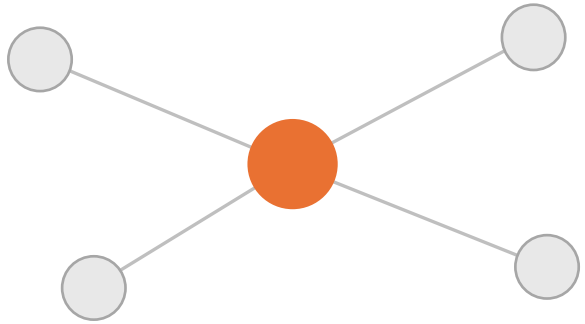
Tre meccanismi

- **CLAUDE.md:** il contratto, sempre caricato (user o team)
- **Auto Memory:** fatti persistenti, letti on-demand (machine-local)
- **Shortcut:** # salva un fatto in memoria al volo

CON NOI È SEMPLICE

Tokensave & Episodic memory

Tokensave



Episodic memory



Tokensave: capire il codice senza leggerlo tutto

- Indicizza una volta la codebase in un knowledge graph locale
- Query strutturate: solo simboli e snippet pertinenti, non interi file

Episodic memory: recuperare il perché

- Indicizza le conversazioni passate (ricerca semantica, locale)
- Ritrova decisioni, alternative scartate e trade-off

Completano le memorie curate.

CON NOI È SEMPLICE

Hands-on

Modulo 4

Plugin

plugin.json

```
{
  "name": "dev-tools",
  "description": "Blazor development utilities",
  "version": "1.0.0",
  "author": {
    "name": "Gerardo Greco",
    "email": "gerardo.greco@blexin.com"
  },
  "license": "MIT",
  "keywords": ["blazor", "frontend"],
  "agents": "agents/",
  "skills": ["skills/", "extra-skills/"],
  "hooks": "hooks.json",
  "mcpServers": ".mcp.json"
}
```

I plugin consentono di estendere GitHub Copilot con estensioni condivisibili tra progetti e team.

CON NOI È SEMPLICE

Marketplace

.github/hooks/hooks.json

```
"userPromptSubmitted": [
  {
    "type": "command",
    "bash": "./scripts/log-prompt.sh",
    "powershell": "./scripts/log-prompt.ps1",
    "cwd": "scripts",
    "env": {
      "LOG_LEVEL": "INFO"
    }
  }
]
```

Un marketplace è un catalogo di plugin creati e condivisi da altri sviluppatori.

CON NOI È SEMPLICE

Hands-on

Modulo 5


THANK YOU